In [62]:
```
import pandas as pd
from scipy.stats import wilcoxon
```

Case, where each thread has <n := number of threads> tasks

In [63]:
```
thread_pool_1 = pd.read_csv("./integral_parallel_measurements/measurement
work_stealing_1 = pd.read_csv("./integral_parallel_measurements/measureme
```

In [64]:
```
columns = ["1", "2", "3"]
results = {}

for col in columns:
    stat, pvalue = wilcoxon(thread_pool_1[col], work_stealing_1[col])
    results[col] = stat, pvalue

for i, values in results.items():
    stats, pvalue = values
    print(f"Column: {i}: Statistics {stats}, P-value: {pvalue}")
```

```
Column: 1: Statistics 50.0, P-value: 0.568889810834584
Column: 2: Statistics 54.0, P-value: 0.4679955610117016
Column: 3: Statistics 6.0, P-value: 0.34411849613531187
```

As in all the cases p-value of the test is very high, we cannot reject two-sided wilcoxon test's null hypothesis (it states that time efficiency of these two approaches should be almost the same).

Each thread has $3n$ tasks

In [65]:
```
thread_pool_3 = pd.read_csv("./integral_parallel_measurements/measurement
work_stealing_3 = pd.read_csv("./integral_parallel_measurements/measureme
```

In [66]:
```
for col in columns:
    stat, pvalue = wilcoxon(thread_pool_3[col], work_stealing_3[col])
    results[col] = stat, pvalue

for i, values in results.items():
    stats, pvalue = values
    print(f"Column: {i}: Statistics {stats}, P-value: {pvalue}")
```

```
Column: 1: Statistics 51.5, P-value: 0.39260052884527485
Column: 2: Statistics 44.5, P-value: 0.22422912586005328
Column: 3: Statistics 13.0, P-value: 0.0747913775869442
```

The same situation is noted, when we assign 3*threads tasks to each worker. However, p-value is much smaller. For third function, if we set size of the test to be 0.1, we even can say that work stealing approach is better.

Each thread has $8n$ tasks

In [67]:
```
thread_pool_8 = pd.read_csv("./integral_parallel_measurements/measurement
work_stealing_8 = pd.read_csv("./integral_parallel_measurements/measureme


for col in columns:
    stat, pvalue = wilcoxon(thread_pool_8[col], work_stealing_8[col])
    results[col] = stat, pvalue

for i, values in results.items():
    stats, pvalue = values
    print(f"Column: {i}: Statistics {stats}, P-value: {pvalue}")
```

```
Column: 1: Statistics 14.0, P-value: 0.015551296117471632
Column: 2: Statistics 16.0, P-value: 0.007160087131374697
Column: 3: Statistics 20.0, P-value: 0.04107690290648811
```

When each thread has substantial amount of jobs, we can see that pvalues are lower than standard size of test 0.05. Test's statistics are also positive, thus we conclude that indeed work stealing approach gives us boost in performance, when we may meet the situation that some worker does not have a job.